RESEARCH MEETING

Tessera: Unlocking Heterogeneous GPUs through Kernel-Granularity Disaggregation

Offline Dependencies, Pipelined Execution, Online Adaptation

Yu-Jui Liang

Advisor: Hsiang-Yun Cheng

National Taiwan University | Academia Sinica | 28 Oct 2026

Tiancheng Hu, Jin Qin, Zheng Wang, Junhao Hu, Yuzheng Wang, Lei Chen, Yizhou Shan, Mingxing Zhang, Ting Cao, Chunwei Xia, Huimin Cui, Tao Xie, Chenxi Wang

arXiv:2604.10180v2, 9 Jul 2026

Presentation Roadmap

TESSERA / OVERVIEW

1. **Background / Motivation**
   Why phase and block splits still misplace kernels.
2. **Methodology**
   PTX dependencies, a throughput MILP, and a latency policy with a monitor.
3. **Experimental Results**
   Five GPU types, four model families, and clusters up to 16 GPUs.
4. **Discussion / Conclusion**
   Cost, generality, and a comparison with ThunderServe, Hetis, and Helix.

Source: Local Tessera PDF. The comparison page is our synthesis, not a table from the paper.

Hardware Heterogeneity Is Kernel-Dependent

Table I. Datasheet rates and relative rental price. Price is normalized to the L40s. Tildes are printed in the paper.

GPU	CUDA FP32	Tensor BF16	L2 (MB)	Mem. (GB)	BW (GB/s)	Price
A100	19.5	312	40	80	1935	1.5
H100	67	989	50	80	3350	2.9
B200	80	∼2500	126	192	∼8000	5.0
L40s	91.6	366.5	96	48	864	1
RTX Pro 6000	120	∼500	126	96	1597	1.2

- Vs. A100, the L40s has 4.7× the CUDA-core rate and 1.2× the tensor-core rate, but 45% of the memory bandwidth.
- The paper says L40s and RTX Pro 6000 reach up to 7.4× higher compute throughput per dollar. It does not spell out the denominator GPU.

No GPU in Table I wins compute, bandwidth, capacity, and price at once. These are not serving measurements.

Source: Sec. II-A, Table I. Prices are "derived mainly" from Google Cloud standard rates.

Why Phase- or Block-Level Disaggregation Is Too Coarse

TESSERA / MOTIVATION

- Normal execution runs K1–K6 on one GPU. Control flow is the black order.
- Disaggregation may put K1, K3, K4 on GPU A and K2, K5, K6 on GPU B.
- Red arrows are data dependencies that used to be local memory reads.
- This drawing is a schematic. It is not a profiled trace.

Splitting kernels is useful only if those red edges stay correct.

Source: Sec. I, Fig. 1. PDF p. 2.

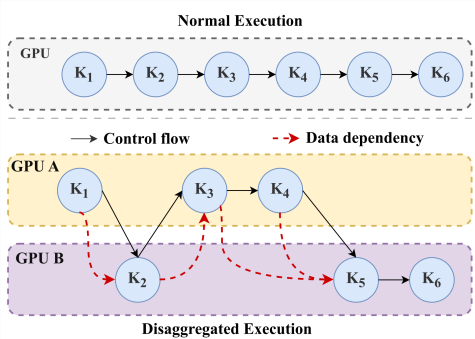


Fig. 1: Simplified example of kernel disaggregation.

Kernel as the Scheduling Granularity

TESSERA / MOTIVATION

TOO FINE

Thread blocks

Inside one kernel, blocks are largely homogeneous. Further splitting does not track a different hardware preference.

TOO COARSE

Phase or block

Prefill vs. decode, or attention vs. FFN, still mixes kernels that prefer different GPUs.

THE UNIT

One GPU kernel

The kernel is the user-level launch Tessera schedules. It is not a new model-layer abstraction.

- Phase and block methods are also tied to a model family. Mamba has no attention block; diffusion has no prefill/decode split.
- Tessera's claim is that the kernel exists in all four families they evaluate.

The granularity argument is architectural and empirical. The counts are on the next two slides.

Source: Sec. I and Sec. II-C.

Different Kernels Prefer Different GPUs

TESSERA / MOTIVATION

- Profiled pair: A100 and L40s. Models: Llama3 8B, GPT-oss 20B, Mamba 7B, Qwen2.5-VL 7B, SD3.5.
- (a) By kernel count, 67% of kernels are faster on the L40s, averaged across models and tasks.
- (b) Weighted by time on the A100, those kernels are 36% of end-to-end time, and up to 53% for diffusion.
- Count and time answer different questions. A numerous short kernel is not 53% of the job.

There is a real L40s-favoring slice. It is not "most of the runtime" on every model.

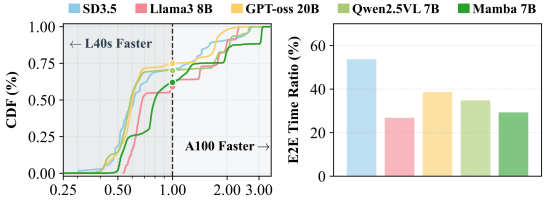


Fig. 2: Kernel heterogeneity between A100 and L40s. (a) CDF of kernel execution time ratio (L40s/A100) based on kernel count, and (b) time-weighted percentage of L40s-faster kernels within the end-to-end (E2E) task execution time on A100.

Source: Sec. II-B, Fig. 2. The motivation text says Mamba 7B; the eval model is Mamba-Codestral 7B.

Phase Labels Hide Kernel-Level Variation

TESSERA / MOTIVATION

- GPT-oss 20B. Points above the line are faster on the A100; points below are faster on the L40s.
- Inside one phase: 45% of prefill kernels and 57% of decode kernels are faster on the L40s.
- cublasGemv is about 1 FLOP/byte. A100's HBM is $\sim 2\times$ L40s, and the kernel is $1.9\times$ faster on A100.
- FlashAttention is compute-heavy. L40s is up to $2.1\times$ faster. Inside blocks: 71% of attention kernels and 33% of FFN kernels prefer the L40s.

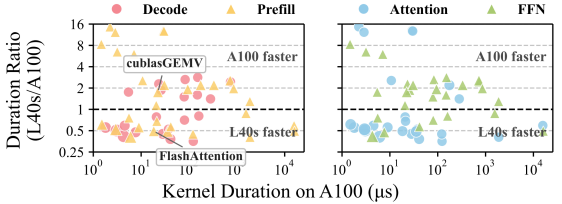


Fig. 3: Kernel duration ratio (L40s / A100) for GPT-oss 20B inference, grouped by (a) prefill and decode phases and (b) attention and FFN blocks. Kernels above the dashed line run faster on A100, while those below run faster on L40s.

A phase label is not a hardware preference. Both phases contain both kinds of kernel.

Source: Sec. II-C, Fig. 3. PDF p. 3. Ratios are L40s duration over A100 duration.

Three Challenges for Kernel Disaggregation

TESSERA / PROBLEM

1 / CORRECTNESS

RAW across GPUs

A kernel on one GPU may read a buffer written on another. Opaque and JIT kernels do not expose that buffer.

2 / BUBBLES

Wait on the dependency

The downstream GPU idles unless another request's ready kernels cover the transfer.

3 / POLICY

Two objectives

Offline jobs want throughput. Online serving wants per-request latency, and the load changes.

- A bad placement can spend more on communication and imbalance than it saves in kernel time.
- Tessera's answer is one analyzer, two MILP objectives, a pipeline, and a windowed monitor.

The rest of the talk is those three answers, in that order.

Source: Sec. I. The three challenges are the paper's own framing.

Tessera System Overview

TESSERA / SYSTEM

- Analyzer: memory ranges, RAW edges, and per-GPU kernel latency.
- Planner: a kernel-to-GPU policy for each dependency graph.
- One worker per GPU runs only its kernels and the cut-edge transfers.
- The monitor feeds runtime latency back to policy selection.

NVLink and RDMA in the figure are the design picture. Measured links are stated later.

Source: Sec. III, Fig. 4. PDF p. 4.

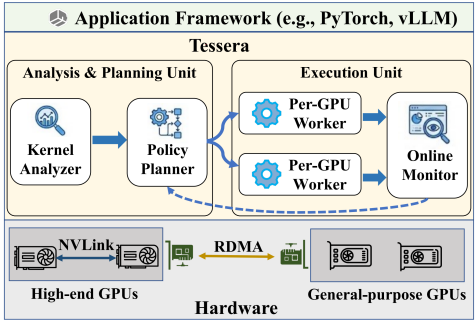


Fig. 4: Design overview of Tessera.

Offline Planning and Online Adaptation

TESSERA / SYSTEM

OFFLINE

Analyzer and planner

Run once, usually during warm-up. Build one dependency graph per CUDA graph and solve its MILP. No per-token replanning.

ONLINE

Monitor

Every window W, compare queueing pressure with a threshold and switch between the two policies.

ONLINE

Workers

Select the precomputed policy when a CUDA graph is launched. Overlap send/recv with compute on separate streams.

NOT IN THE MILP

GPU memory

The formulation does not constrain capacity. Weights are replicated, as in prefill/decode disaggregation.

The expensive graph and MILP stay off the request path. The monitor only chooses which stored policy to run.

Source: Sec. III and Sec. IV. Memory replication is stated in Sec. IV.

Analyzer: Standard vs. Opaque Kernels

TESSERA / ANALYZER

STANDARD

Library kernels

cublasSgemm reads A and B and writes C. Sizes come from the API arguments. No instrumentation is required to know the buffers.

OPAQUE

User or JIT kernels

Semantics are not available when the framework launches them. Tessera instruments PTX instead of guessing API arguments.

WHY NOT API GUESSING

Caching allocator

PhoenixOS can see base addresses, but PyTorch's allocator does not call CUDA alloc on every tensor. The guessed size is often too large.

COST OF A LARGE GUESS

Extra traffic

An overestimated buffer is still transferred. Tessera keeps only the addresses the kernel actually touches.

Standard kernels are read from their API. Opaque kernels are read from instrumented PTX.

Source: Sec. III-A. PhoenixOS is cited as prior buffer discovery, not as a Tessera component.

PTX Instrumentation and Address Ranges

TESSERA / ANALYZER

1. Add an instrumentation buffer to the kernel parameters and load it with `ld.param`.
2. Around each `ld.global` / `st.global`, emit `atom.global.min` and `atom.global.max` so threads agree on one range.
3. After the launch, align each instruction's $[\min, \max]$ interval with buffer base addresses. The opcode says read or write.
4. Recompile the patched PTX with `ptxas`. The probe engine is built on NEUTRINO.

If a kernel touches a global that is not a launch argument, the base address is unknown. Tessera then refuses to disaggregate that kernel. The paper cites a study that this pattern is absent in PyTorch and vLLM, and present in one Rodinia kernel.

The analyzer records touched address ranges. It does not reconstruct a kernel's arithmetic.

Source: Sec. III-A and Sec. IV. Listing 1 is the paper's instrumentation sketch.

Extracting RAW Dependencies

TESSERA / ANALYZER

- Execution order is preserved, so only Read-After-Write edges can change the result. WAR and WAW are not the edges Tessera schedules.
- A registry stores the last writer of each buffer. A later reader adds an edge from that writer, annotated with the measured byte size.
- Asynchronous copies are nodes too. Their source, destination, and size come from the API.
- One CUDA graph gets one dependency graph. Dynamic shapes become several graphs, not one graph with a runtime branch.
- Frameworks without CUDA Graph can record a trace and replay it under stream capture. That tool is preprocessing, not the online scheduler.

The graph is a static RAW graph per captured execution, plus the byte size of each edge.

Source: Sec. III-A. “DDG” is the paper’s term for this graph.

Cross-Iteration Dependencies and KV State

TESSERA / ANALYZER

- The default graph is intra-iteration. KV cache is written in one iteration and read in a later one.
- The analyzer flags those buffers by profiling more than one iteration.
- At runtime each GPU keeps a replica. Only the delta is copied, asynchronously.
- Two NCCL communicators separate this background copy from the latency-critical cut-edge transfers. Dependency traffic gets the higher class.
- The paper says delta replication is under 0.1% of latency, citing prior prefill/decode work rather than a new Tessera measurement.

KV state is replicated and updated in the background. It is not a cut edge on the per-iteration critical path.

Source: Sec. III-A and Sec. IV. The 0.1% figure is attributed to prior work [9], [10].

Planner Inputs, Decisions, and Objectives

Input	Meaning
G, K, E	GPUs, kernels, and RAW edges
$t_{k,g}$	Profiled latency of kernel k on GPU g
d_{ij}	Bytes on the edge from kernel i to j
$bw_{u,g}$, $\ell_{u,g}$	Bandwidth and base latency between GPUs u and g

- Decision: binary $x_{k,g}$. Each kernel is placed on exactly one GPU.
- Throughput objective, for offline batches: minimize the slowest GPU's stage time, assuming communication can overlap computation.
- Latency objective, for a lightly loaded online request: minimize kernel time plus transfer time along the cut, with no overlap credit.

Same graph, two objectives. The monitor chooses which solved policy to run.

Source: Sec. III-B, Table II. Symbols are the paper's; the overlap remark is the surrounding prose.

Offline Throughput-Oriented MILP

$$\sum_{g \in G} x_{k,g} = 1, \qquad T_g = \sum_{k \in K} t_{k,g}\, x_{k,g}, \qquad W_g = \max(T_g, M_g), \qquad \min \max_{g \in G} W_g.$$

- T_g is compute time on GPU g. M_g, on the next slide, is communication time into g.
- W_g is the slower of the two. Steady-state throughput is then one request per $\max_g W_g$.
- This is the offline objective. It is the wrong objective when too few requests are in flight to hide transfers.
- Gurobi solves it offline. A reported case, GPT-oss 20B on A100+L40s, has about 500 kernels and finishes one graph in about 20 ms.

The MILP balances GPUs by minimizing the slowest stage. It does not maximize a sum of kernel speeds.

Source: Sec. III-B. The 20 ms figure is for $|G| = 2$, $|K| \approx 500$; all patterns take about 5 s.

Communication and Load-Balancing Constraints

TESSERA / PLANNER

$$c_{ij}^{u,g} = \ell_{u,g} + \frac{d_{ij}}{bw_{u,g}}, \qquad M_g = \sum_{(i,j)\in E} \sum_{u\in G\setminus\{g\}} c_{ij}^{u,g}\, y_{ij}^{u,g}.$$

- $y_{ij}^{u,g} = x_{i,u}x_{j,g}$ is 1 when the producer is on u and the consumer is on g. The product is linearized with the standard Boolean replacement.
- Load balance is not a separate variance constraint. Minimizing $\max_g W_g$ already penalizes the busiest GPU.
- Per-GPU memory capacity is absent. At initialization every GPU receives a full copy of the weights.
- Offline, once the policy is fixed, unused weight buffers can be reclaimed. The paper does not report that reclamation in the evaluation tables.

Communication is bytes over profiled bandwidth, plus a base latency. Memory is a runtime caveat, not a constraint.

Source: Sec. III-B and Sec. IV. Linearization cites Nemhauser and Wolsey.

Online Latency-Oriented Scheduling

TESSERA / PLANNER

Under low load the pipeline cannot hide transfers. The objective adds kernel time and cut-edge transfer time:

$$\min \sum_{k,g} t_{k,g} x_{k,g} + \sum_{(i,j)} \sum_{u} \sum_{g \neq u} c_{ij}^{u,g} \, y_{ij}^{u,g}.$$

- There is no $\max(T, M)$ here. Overlap is not assumed.
- The same Gurobi solve is offline. Workers only look up the stored policy.
- If communication is expensive, this objective keeps dependent kernels on the same GPU. Fig. 11 shows that behavior at 25 Gb/s.

Latency mode shortens one request's critical path. It is not the throughput MILP with a different label.

Source: Sec. III-B. The sum is the paper's low-load decomposition, compacted for the slide.

Pipelining Across Requests

- Without a pipeline, a GPU that waits on a remote buffer sits idle. The paper's later drill-down finds about half the optimal throughput in that mode.
- Each in-flight request gets its own CUDA stream. A blocked stream does not stop another request's ready kernels.
- Equal-priority streams tend to reach their transfers together, so the GPU still goes idle.
- Later requests get a lower CUDA stream priority. The hardware scheduler then prefers earlier requests and staggers the transfers.
- "Optimal" in that drill-down means zero communication overhead, not a better kernel algorithm.

The pipeline hides transfers only when several requests are in flight and they are not synchronized.

Source: Sec. III-C. The half-throughput and priority results are measured in Sec. V-D, Fig. 9.

GPU Worker Execution

TESSERA / EXECUTION

- Fig. 5 is GPU B from the six-kernel example. Compute and communication use different streams.
- R_{ij} receives kernel i's output for kernel j. S_{ij} sends it. CUDA events order the two streams.
- Receives are posted before the consumer kernel. Sends are posted after the producer if the edge is cut.
- Sends and receives are GPU-side kernels via NCCL's GPUDirect Async (IBGDA), so the CPU is not on the critical path.

A worker runs its own kernels and the cut-edge messages. It does not run the whole model.

Source: Sec. III-C, Fig. 5. PDF p. 6.

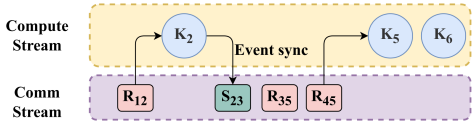


Fig. 5: An execution example of the GPU worker. Compute kernels and communication operations run on separate streams, synchronized via CUDA events. R_{ij} denotes receiving the output of kernel i for kernel j, S_{ij} denotes sending data from kernel i to kernel j.

Overlapping Communication and Computation

TESSERA / EXECUTION

- Tessera intercepts CUDA Graph creation, partitions nodes onto GPUs, and embeds send/recv kernels in each subgraph.
- Events that order the compute and communication streams become ordinary edges inside that subgraph.
- Each worker keeps a table from the original graph handle to the policy and the subgraphs. Launch picks the row; it does not rebuild the graph.
- Collectives of an existing tensor-parallel group stay on the original homogeneous GPUs. Tessera only reschedules compute kernels inside a heterogeneous pair.
- That composition is how the 8 B200 + 8 H100 experiment is run. It is not a claim that every collective was rewritten.

Graph replay is the fast path. The MILP has already finished before the first replay.

Source: Sec. III-C and Sec. IV. TP composition is also used in Sec. V-C.

Online Monitor Signals

TESSERA / RUNTIME

- The monitor records per-request latency, communication time, and per-kernel-group latency.
- A kernel group is the run of kernels between two consecutive communication operations. Individual kernels are not timed online.
- $\bar{L}_{\mathrm{req}}$ is average end-to-end request latency. $\bar{L}_{\mathrm{exec}}$ sums compute and communication and excludes queueing.
- The pressure signal is the ratio $\bar{L}_{\mathrm{req}}/\bar{L}_{\mathrm{exec}}$.
- A low ratio means little queueing, so the latency policy is preferred. A high ratio means queueing dominates, so the throughput policy is preferred.

The switch looks at queueing pressure, not at a hand-named GPU counter.

Source: Sec. III-D. Kernel group is the paper's definition.

Policy Switching and Sensitivity Parameters

TESSERA / RUNTIME

WINDOW W

How often to decide

The ratio is computed at window boundaries. The chosen defaults are $W = 300$ ms and $\beta = 1.5$.

SWITCH COST

About 30 ms

Workers synchronize on an iteration boundary. The 30 ms is the paper's reported stall, not a bound.

THRESHOLD β

When queueing wins

Crossing β selects the throughput policy. Staying below it selects the latency policy.

WHERE IT WAS TUNED

One trace

GPT-oss 20B, 360 s of the Azure Conversation trace. Fig. 10 is the sensitivity plot.

W and β are the only two knobs the paper names. Aggressive and lazy settings both increase latency.

Source: Sec. III-D and Sec. V-D. Numeric penalties are on the sensitivity slide.

Implementation Stack

TESSERA / IMPLEMENTATION

GPU WORKER

About 3k lines

C++ and CUDA. Integrated with vLLM and PyTorch. Evaluation uses vLLM v0.18.0.

SOLVER

Gurobi, offline

One graph at $|G| = 2$, $|K| \approx 500$: about 20 ms. Enumerating patterns: about 5 s. Not on the request path.

ANALYZER

About 2k lines

Python. PTX injection, RAW construction, and the preprocessing capture tool.

FALLBACK

No disaggregation

Indirect global accesses that are not launch arguments stay on one GPU. The paper treats this as rare for PyTorch and vLLM.

The line counts are the paper's. They are not a complexity comparison with Helix.

Source: Sec. IV and the solver paragraph in Sec. III-B.

Communication Backends and Interconnects

TESSERA / IMPLEMENTATION

- The communication backend in the evaluation is NCCL. Cut-edge sends use GPU-initiated IBGDA.
- Local pairs: A100+L40s, H100+RTX Pro 6000, and B200+H100. Each GPU has a 200 Gb/s RDMA NIC. The paper does not say these pairs use NVLink.
- Cluster, inside a homogeneous high-end group: NVLink. Across nodes: 200 Gb/s for 2 A100 + 1 L40s, and 400 Gb/s for 8 B200 + 8 H100.
- Fig. 4 draws NVLink between high-end GPUs and RDMA toward general-purpose GPUs. That is the design diagram, not a per-experiment wiring list.
- The paper explicitly does not evaluate AMD GPUs, NPUs, PCIe-only hosts, or Ethernet.

Say NVLink only for the homogeneous group inside the cluster experiments.

Source: Sec. IV and Sec. V-A. “Respectively” in the bandwidth sentence maps onto the two cluster setups.

Evaluation Methodology

TESSERA / EVALUATION

Family	Model	Offline data
LLM	Llama-3 8B, GPT-oss 20B	Splitwise conversation; median 1020 in / 129 out
SSM	Mamba-Codestral 7B	Same conversation trace
MLLM	Qwen2.5-VL 7B	COCO captions, images resized to 512×512
Diffusion	Stable Diffusion 3.5	PartiPrompts, 1024×1024, 28 denoising steps

- Baselines: homogeneous left and right GPUs; PD follows DistServe; AF follows MegaScale-Infer. PD does not apply to diffusion. AF does not apply to SSM or diffusion.
- Engine: vLLM v0.18.0, continuous batching on, DVFS off. Each run is repeated "multiple times"; variance is below 1%.
- Throughput is output tokens/s for LLM, SSM, and MLLM. Diffusion is images/min on Fig. 6, which the text also calls requests per minute.
- Online experiments are GPT-oss 20B only: Poisson arrivals, plus one Azure trace for the monitor.

A red X on Fig. 6 means the baseline does not apply. It is not a zero-throughput measurement.

Source: Sec. V-A. Normalized online latency is end-to-end time divided by output tokens.

Offline Throughput

TESSERA / RESULTS

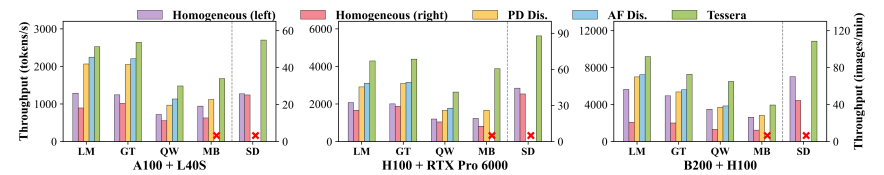


Fig. 6: Offline throughput across five workloads and three heterogeneous GPU pairs. Left y-axis: tokens/s for LLM, MLLM, and SSM workloads, right y-axis: images/min for SD. Red X marks indicate that the disaggregation method is inapplicable.

Fig. 6. Three pairs. Left axis: tokens/s. Right axis, SD only: images/min. Red X: method does not apply.

- Against PD disaggregation: 1.5× on average, up to 2.3×. Against AF disaggregation: 1.35× on average, up to 1.7×.
- The paper does not assign those maxima to one named bar. Do not read a single panel as "the" 2.3×.
- Gains are attributed to scheduling kernels that a phase or a block would have kept together.

The 2.3× is the largest reported ratio against PD here, not a typical per-request speedup.

Source: Sec. V-B, Fig. 6. PDF p. 8. Five workloads times three pairs; some baseline bars are absent.

Online Latency under a Token SLO

TESSERA / RESULTS

- GPT-oss 20B, Poisson arrivals, three pairs. Y-axis: end-to-end latency per output token.
- The red line is the SLO, 50 ms/token. This is not DistServe's goodput definition.
- At low rate, Tessera's normalized latency is 1.3× and 1.2× lower than PD and AF.
- Under the SLO, Tessera holds up to 1.3× the request rate of the best baseline.
- The abstract's "1.3× more requests under the same SLO" is this comparison.

Low-rate latency and high-rate SLO capacity are both reported. They are different operating points.

Source: Sec. V-B, Fig. 7. PDF p. 8. Online scope is this one LLM.

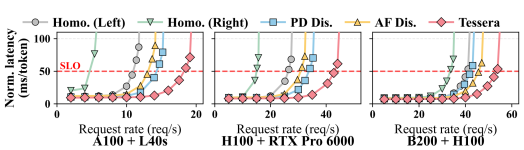


Fig. 7: Online normalized latency on GPT-oss 20B across three GPU pairs. The red dashed line marks the SLO target (50 ms/token).

Scaling with Cluster Size

TESSERA / RESULTS

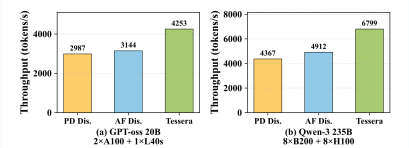


Fig. 8: Cluster-scale offline throughput. (a) GPT-oss 20B on 2 A100 + 1 L40s. (b) Qwen-3 235B on 8 B200 + 8 H100.

Fig. 8. Offline tokens/s. (a) 2 A100 + 1 L40s. (b) 8 B200 + 8 H100, kernel split inside each TP rank.

Setup	PD	AF	Tessera
GPT-oss 20B, 2 A100 + 1 L40s	2987	3144	4253
Qwen-3 235B, 8 B200 + 8 H100	4367	4912	6799

PD on (a) uses the L40s for prefill and each A100 as a decode instance. AF runs attention on the two A100s at TP = 2 and FFN on the L40s. The paper summarizes both setups as 1.5× vs. PD and 1.4× vs. AF.

These are offline cluster throughputs. Qwen-3 235B is not in the single-pair Fig. 6 suite.

Source: Sec. V-C, Fig. 8. PDF p. 9. Units are tokens/s.

Pipeline Effectiveness

TESSERA / DRILL-DOWN

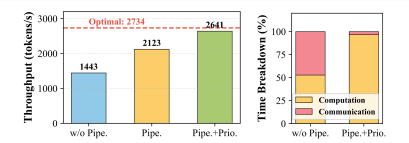


Fig. 9: (a) Throughput under different pipeline configurations.
(b) Time breakdown on the bottleneck GPU.

Fig. 9. GPT-oss 20B, offline. The caption does not name the GPU pair.

- No pipeline: 1443 tokens/s, about half of the 2734 tokens/s zero-communication reference.
- Pipeline: 2123 tokens/s, 1.47× the no-pipeline bar. Priority: 2641 tokens/s, 96.6% of that reference.
- On the bottleneck GPU, communication idle time falls from nearly half to under 4%.
- 2641 tokens/s matches Fig. 11's 200 Gb/s A100+L40s bar. That match is our cross-figure reading, not a caption statement.

Priority matters because it keeps transfers from lining up. The model does not assume the idle time is zero.

Source: Sec. V-D, Fig. 9. PDF p. 9.

Online Monitor Sensitivity

TESSERA / DRILL-DOWN

- GPT-oss 20B, 360 s Azure trace. Left: sweep W at $\beta = 1.5$. Right: sweep β at $W = 300$ ms.
- $W = 30$ ms or $\beta = 1.1$: frequent switches, latency up to $1.32\times$.
- $W = 1.5$ s or $\beta = 3.0$: slow reaction, latency up to $1.55\times$.
- Each switch stalls workers for about 30 ms at an iteration boundary.
- The chosen point is $W = 300$ ms, $\beta = 1.5$. It is the best tradeoff in this sweep, not a universal constant.

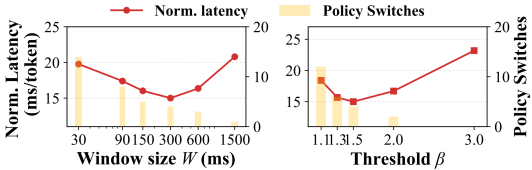


Fig. 10: Sensitivity of normalized latency and policy switch count to window size W (β=1.5) and queueing threshold β (W=300ms) on GPT-oss 20B.

Both too many switches and too few switches raise latency on this trace.

Source: Sec. V-D, Fig. 10. PDF p. 10.

Network Bandwidth and MILP Analysis

TESSERA / DRILL-DOWN

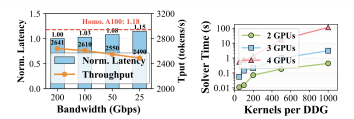


Fig. 11: (a) Impact of network bandwidth on GPT-oss 20B with A100+L40s. (b) MILP solver time.

Fig. 11. (a) GPT-oss 20B, A100+L40s. (b) Gurobi time vs. kernels per graph.

(a) Throughput at 200 / 100 / 50 / 25 Gb/s: 2641 / 2610 / 2550 / 2490 tokens/s. The 25 Gb/s drop is under 6%. Normalized latency: 1.00 / 1.03 / 1.08 / 1.15, still below the homogeneous A100 line at 1.18.

(b) Text, 2 GPUs: 0.07 s at 200 kernels and 0.43 s at 1000 kernels. A DeepSeek-V3 671B forward graph of about 1500 kernels solves within 1 s. That is a solver stress test, not a serving result. Time grows with the number of GPU types; repeated layers are solved once.

At 25 Gb/s the offline pipeline still hides most of the extra delay. The latency policy moves work off the slow cut.

Source: Sec. V-D, Fig. 11. PDF p. 10. Do not read panel (b) as an end-to-end DeepSeek result.

Cost Efficiency of Heterogeneous Pairing

TESSERA / RESULTS

Table III. Perf/$: mean throughput over the five workloads, divided by rental cost, then normalized to the left GPU alone. A pair pays for both GPUs.

System	A100+L40s	H100+RTX Pro 6000	B200+H100
Homogeneous left	1.00	1.00	1.00
Homogeneous right	1.18	2.01	0.78
PD disaggregation	0.87	1.00	0.70
AF disaggregation	1.02	1.07	0.74
Tessera	1.21	1.64	1.01

- Vs. PD: 1.5× cost efficiency on average, up to 1.6×. Vs. AF: 1.4× on average, up to 1.5×.
- On H100+RTX Pro 6000, Tessera is 1.64× the Perf/$ of H100 alone and 2.3× its throughput, above two H100s, at price $2.9 + 1.2$ versus 2×2.9.

The pair can beat a single expensive GPU on Perf/$. The prices are Table I's relative rentals, not a new quote.

Source: Sec. V-B, Table III. The 2.3× sentence is the cost-efficiency paragraph, not the PD maximum.

Generality Across Model Families

TESSERA / RESULTS

What was run	Models	What it does not show
Offline pairs	Llama-3 8B, GPT-oss 20B, Qwen2.5-VL 7B, Mamba-Codestral 7B, SD 3.5	PD or AF is inapplicable on some families
Online SLO	GPT-oss 20B only	Latency for SSM, MLLM, or diffusion
Cluster, offline	GPT-oss 20B and Qwen-3 235B	The single-pair Fig. 6 suite
Solver stress test	DeepSeek-V3 671B, one forward graph	End-to-end serving

- AF needs an attention block, so Mamba and diffusion are out. PD needs a prefill/decode split, so diffusion is out.
- The motivation profiles say "Mamba 7B"; the evaluated SSM is Mamba-Codestral 7B. Do not merge the two names.

"Four families" means the offline pair study. Online evidence is one LLM.

Source: Sec. II-B, Sec. V-A–V-D. DeepSeek-V3 is the solver paragraph in Sec. V-D.

Tessera vs. ThunderServe, Hetis, and Helix

CROSS-PAPER COMPARISON

	ThunderServe	Hetis	Helix	Tessera
Granularity	Prefill / decode phase	Dense module vs. head	Layer range and path	CUDA kernel
Decision	Group, phase, route	Primary set, then heads	Placement, then a path	Kernel-to-GPU map
Time scale	Offline + light reschedule	Offline + online re-dispatch	Once per cluster + per request	Offline MILP + window W
State moved	KV at phase boundary	QKV and KV shards	Tokens and activations	Cut tensors; KV delta
Adaptation	Phase role, failure	Re-dispatch, migration	KV mask; path reused	Switch on $\bar{L}_{\text{req}}/\bar{L}_{\text{exec}}$
Environment	Heterogeneous cloud	Local heterogeneous cluster	Heterogeneous, incl. geo	NVIDIA pairs; NVLink + RDMA
Models	LLM	LLM	LLaMA 30B / 70B	LLM, SSM, MLLM, diffusion
Main limit	Heuristic phase search	Head groups; local net	One-shot plan; geo simulated	No memory constraint in MILP

Kernel splitting is a different cut from phase, head, or layer placement. Do not rank the papers by their largest ratio.

Source: Our synthesis. Tessera cells are this PDF. Other cells come from each system's own paper.

Limitations and Threats to Validity

TESSERA / DISCUSSION

- The MILP ignores per-GPU memory. Full weight replication is conservative, and the paper does not show reclamation in the result tables.
- Online latency, the SLO, and the monitor sweep are GPT-oss 20B. Diffusion and Mamba are offline pair results.
- The $< 0.1\%$ delta-replication cost is cited from prior PD work. The $< 4\%$ idle time is Tessera's own Fig. 9.
- Indirect PTX accesses fall back to one GPU. The paper limits the study to NVIDIA.
- Fig. 9's caption does not name the GPU pair. A policy switch stalls about 30 ms.
- There is no shared-hardware comparison against Helix, Hetis, or ThunderServe.

Generality is real for offline kernels. It is not yet a latency result for every family.

Source: Sec. IV, Sec. V, and Sec. VII. The cross-paper gap is ours, not a claim in Tessera.

Conclusion

TESSERA / CONCLUSION

1

Schedule kernels

Phase and block labels still mix kernels that prefer different GPUs. The kernel is the unit Tessera places.

2

Make the split correct

PTX ranges build a RAW graph. The throughput MILP overlaps compute with communication; the latency MILP does not.

3

What was measured

Up to 2.3× throughput vs. PD and up to 1.6× Perf/$. A pair can beat two H100s on the reported H100+RTX setup.

- The same system serves models where PD or AF does not apply, in the offline pair study.
- The monitor's 1.3× SLO result is GPT-oss 20B at 50 ms/token, not a goodput number under another paper's definition.

Tessera treats GPU diversity as something to schedule per kernel, with an offline plan and a small online switch.

Source: Sec. VII, with ratios from Sec. V-B rather than from the abstract alone.

Questions & Discussion

Kernel graph / two MILP objectives / what was actually served

Tessera (kernel) next to phase, head, and layer-pipeline systems